

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 06 -

KUBERNETES AVANÇADO: PERSISTÊNCIA, CONFIGURAÇÃO E ESCALA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMANIP

O QUE VEM POR AÍ?

Nesta aula, vamos aprofundar os três pilares que costumam separar um deploy funcional de uma operação robusta em Kubernetes: persistência de dados, gestão de configuração segura e escala automática.

Esses elementos são essenciais quando o serviço de inferência deixa de ser piloto e passa a operar com carga e SLA reais.

Você vai compreender como PV/PVC, ConfigMaps/Secrets e HPA se conectam em uma arquitetura de ML orientada à confiabilidade, eficiência de custo e governança técnica.

HANDS ON

No hands on, veremos um cenário de inferência com armazenamento persistente para dados de apoio, configuração externa por ambiente e segredo de acesso protegido. O objetivo é mostrar como esses recursos funcionam em conjunto dentro do ciclo de vida do serviço.

Na sequência, você aplicará autoscaling com base em métricas para observar como o cluster responde a aumento de carga. A prática enfatiza a leitura de sinais operacionais para evitar escalabilidade “cega”, que melhora um indicador e piora custo ou estabilidade.

Ao final, teremos um conjunto de decisões técnicas para equilibrar segurança, desempenho e eficiência operacional em workloads de ML com exigência de produção contínua.

SAIBA MAIS

Em estágios iniciais, um serviço de inferência pode operar com configuração simples e escala manual. À medida que a carga cresce, esse modelo se torna frágil. Persistência, configuração segura e escala automática passam a ser requisitos de confiabilidade.

O Kubernetes oferece abstrações maduras para esse cenário, mas o valor aparece quando elas são combinadas com critérios claros de arquitetura e observabilidade.

Persistência com PV/PVC: desacoplamento de estado

Persistent Volumes representam capacidade de armazenamento; PVCs representam a demanda da aplicação. Essa separação permite trocar tecnologia de storage sem alterar o código da aplicação.

Para ML, persistência costuma ser necessária em auditoria, cache de artefatos e resultados críticos que não podem ser perdidos com recriação de Pod.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: ml-inferencia-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard
```

Código-fonte 1 – Exemplo de PVC para armazenar logs/artefatos de inferência (YAML)

Fonte: Elaborado pelo autor (2026)

Persistir tudo aumenta custo e complexidade. Persistir pouco aumenta risco de perda de evidência e quebra de compliance. A decisão deve ser orientada por criticidade de negócio.

TIPO DE DADO	PERSISTÊNCIA RECOMENDADA	MOTIVO
Log de auditoria	Sim	Rastreabilidade legal/técnica
Cache temporário	Depende	Pode ser recomputado
Resultado de decisão crítica	Sim	Impacto direto no negócio
Arquivo intermediário descartável	Não	Baixa criticidade

Tabela 1 – O que persistir e o que manter efêmero
Fonte: Elaborado pelo autor (2026)

ConfigMaps e Secrets: separação disciplinada

ConfigMaps armazenam parâmetros não sensíveis; Secrets armazenam credenciais e tokens. Misturar os dois aumenta risco de exposição e reduz governança.

Em ambientes maduros, configuração e segredo possuem fluxos diferentes de aprovação e rotação.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ml-app-config
data:
  MODEL_NAME: recomendador-v3
  LOG_LEVEL: INFO
---
apiVersion: v1
kind: Secret
```

```
metadata:
  name: ml-app-secret
type: Opaque
stringData:
  API_TOKEN: "token-super-seguro"
```

Código-fonte 2 – Exemplo de ConfigMap e Secret usados pela mesma aplicação (YAML)
Fonte: Elaborado pelo autor (2026)

Secret não é objeto estático: precisa de ciclo de vida (criação, rotação, revogação e auditoria). Além disso, é crucial evitar vazamento indireto em logs e mensagens de erro.

Mini-case 1 - rotação sem indisponibilidade: um serviço migrou para estratégia de segredo externo com rotação programada. O processo reduziu o risco de credencial exposta e manteve disponibilidade por atualização controlada de Pods.

Escala horizontal orientada por métricas relevantes

O HPA é poderoso, mas configuração superficial pode causar oscilação e custo excessivo. Em inferência, escalar só por CPU costuma ser insuficiente para preservar experiência do usuário.

Sempre que possível, combine sinais de latência, throughput e filas para decisões de escala mais fiéis à realidade.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ml-inferencia-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: recomendacao-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 65
```

Código-fonte 3 – Exemplo de Horizontal Pod Autoscaler (YAML)
Fonte: Elaborado pelo autor (2026)

Startup de modelo e eficiência de autoscaling

Um erro comum é assumir que escalar significa apenas “aumentar número de Pods”. Em inferência de ML, isso raramente é tão simples.

Quando um novo Pod sobe, ele pode precisar de tempo para baixar artefatos, carregar pesos em memória, inicializar bibliotecas e aquecer cache. Durante esse intervalo o Pod existe, mas ainda não ajuda de verdade a absorver tráfego.

Por isso, três elementos devem ser pensados juntos:

- Startup probe: protege o processo de boot sem acionar reinícios prematuros.
- Readiness probe: só libera tráfego quando a aplicação realmente está pronta.
- Footprint do modelo: influencia tempo de inicialização e consumo de memória.

De forma didática: um Pod “em Running” não é automaticamente um Pod “pronto para atender”. A eficiência do autoscaling depende de quanto tempo leva para transformar capacidade provisionada em capacidade útil.

Em cargas com picos curtos, esse detalhe é crítico: se o tempo de startup for longo, o autoscaling pode reagir, mas tarde demais para proteger a experiência do usuário naquele pico específico.

Custos de infraestrutura e qualidade de serviço

Escala agressiva sem critério tende a resolver latência no curto prazo, mas pode explodir custo de cluster. Escala conservadora demais reduz gasto imediato, mas degrada SLA, aumenta timeout e pressiona a equipe com incidentes frequentes. O trabalho de engenharia está no equilíbrio entre esses extremos.

Uma forma prática de pensar:

- Defina metas mínimas de serviço (ex.: latência, erro máximo, disponibilidade).
- Observe custo por janela de tempo e por volume de requisições.
- Ajuste limites de escala de forma incremental, nunca em saltos cegos.

Em ML, custo e desempenho também dependem do próprio modelo. Um modelo mais leve pode reduzir necessidade de réplicas; um modelo mais pesado pode exigir tuning de infraestrutura para manter a mesma latência. Por isso, otimização de modelo e tuning de Kubernetes precisam caminhar juntos, não em silos separados.

Quando o time mede apenas custo, perde qualidade. Quando mede apenas latência, perde eficiência financeira. A visão madura é sempre custo e qualidade de serviço ao mesmo tempo.

Observabilidade para ajuste fino de capacidade

Sem observabilidade, ajuste de capacidade vira tentativa e erro. Com observabilidade, ajuste vira engenharia orientada por evidência.

Para esse tipo de decisão, monitore no mínimo:

- Latência (experiência real do usuário).
- Taxa de erro (sinal de saturação ou instabilidade).
- Eventos de escalonamento (quando e por que o HPA acionou).
- Utilização por réplica (CPU, memória, throughput por pod).
- Tempo de prontidão de novos pods (impacto de startup no pico).

Um ponto didático importante: olhar só média de CPU quase sempre mascara o problema. Em inferência, a cauda de latência (p95/p99) geralmente revela gargalos que não aparecem em métricas agregadas simples.

Com esse conjunto de sinais, a equipe consegue responder perguntas objetivas:

- Estamos escalando cedo o suficiente?
- Estamos escalando demais?

- O gargalo é de infraestrutura, aplicação ou modelo?
- Precisamos de mais réplicas, mais recurso por pod ou otimização no pipeline de inferência?

Mini-case 2 - autoscaling sem latência vira armadilha: uma equipe configurou HPA usando apenas CPU média e percebeu que, mesmo com escalonamento frequente, os usuários continuavam relatando lentidão em horários de pico. O diagnóstico mostrou que os Pods novos demoravam para ficar prontos e a métrica de CPU não refletia bem a fila de requisições acumuladas.

O plano de correção foi feito em três frentes:

- Ajuste de readiness/startup para representar melhor o tempo real de inicialização.
- Inclusão de métrica mais aderente ao comportamento da aplicação (fila/latência).
- Revisão de limites de escala para reduzir oscilação excessiva.

Resultado: a latência de cauda caiu, a estabilidade aumentou e o custo mensal ficou mais previsível, porque o cluster deixou de “escalar e desescalar” sem ganho real de experiência.

Lição principal do caso: autoscaling não deve ser guiado por conveniência técnica (apenas CPU), mas por sinais que representem de fato a qualidade percebida pelo usuário.

Erros comuns de adoção e mitigação

Erro comum 1: usar volume efêmero para dado crítico.

Correção: adotar PVC com retenção compatível com risco.

Erro comum 2: tratar segredo como configuração comum.

Correção: separar ConfigMap e Secret com política de rotação.

Erro comum 3: escalar sem critério de experiência do usuário.

Correção: alinhar HPA a métricas de latência e throughput.

Persistência, configuração e escala são pilares interdependentes. Quando bem implementados, sustentam operação de inferência com segurança, previsibilidade e eficiência de custo em cenários reais de produção.

EMSE

MERCADO, CASES E TENDÊNCIAS

No mercado, a discussão sobre Kubernetes avançado deixou de ser “como subir pods” e passou a ser “como operar com previsibilidade e custo sustentável”. Empresas com múltiplos serviços de dados estão padronizando persistência, configuração e escala por políticas declarativas, justamente para reduzir erro humano e acelerar reprodução de ambientes entre times e regiões.

Um caso real frequentemente citado é o do Spotify, que descreve ganhos relevantes de eficiência após evoluir sua operação para Kubernetes, incluindo melhor uso de infraestrutura e maior velocidade de provisionamento. O estudo oficial está [aqui](#).

Para esta aula, o ponto central do case é observar que a escala bem-sucedida não veio apenas de “mais recursos”, mas de um melhor desenho de plataforma e automação operacional.

No recorte de MLOps, a tendência atual é combinar autoscaling orientado por métricas de experiência (latência/throughput), gestão de segredos com integração a cofres corporativos e padrões de persistência reutilizáveis por tipo de workload. Em outras palavras: a maturidade operacional está migrando de decisões artesanais para contratos técnicos claros, monitoráveis e auditáveis

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você consolidou os três pilares que sustentam a operação avançada de inferência em Kubernetes: persistência, configuração segura e escala orientada por métricas. Entendeu como PV/PVC desacoplam estado do ciclo de vida dos Pods, como ConfigMaps e Secrets organizam parâmetros e credenciais com governança e como o HPA pode ajustar a capacidade de forma mais inteligente quando calibrado com sinais que realmente representam experiência do usuário.

Também aprofundou o olhar sobre trade-offs reais de produção: custo versus qualidade de serviço, tempo de startup versus eficiência de autoscaling e decisão baseada em evidência versus tentativa e erro. Em termos práticos, isso significa que você agora possui repertório para sair de uma implantação funcional e evoluir para uma operação robusta, previsível e sustentável para workloads de ML em escala.

REFERÊNCIAS

ALDRED, J. **The Path to Kubernetes**. Boston: O'Reilly Media, 2023.

BURNS, B.; BEDA, J.; HIGHTOWER, K. **Kubernetes: Up and Running**. Sebastopol: O'Reilly Media, 2022.

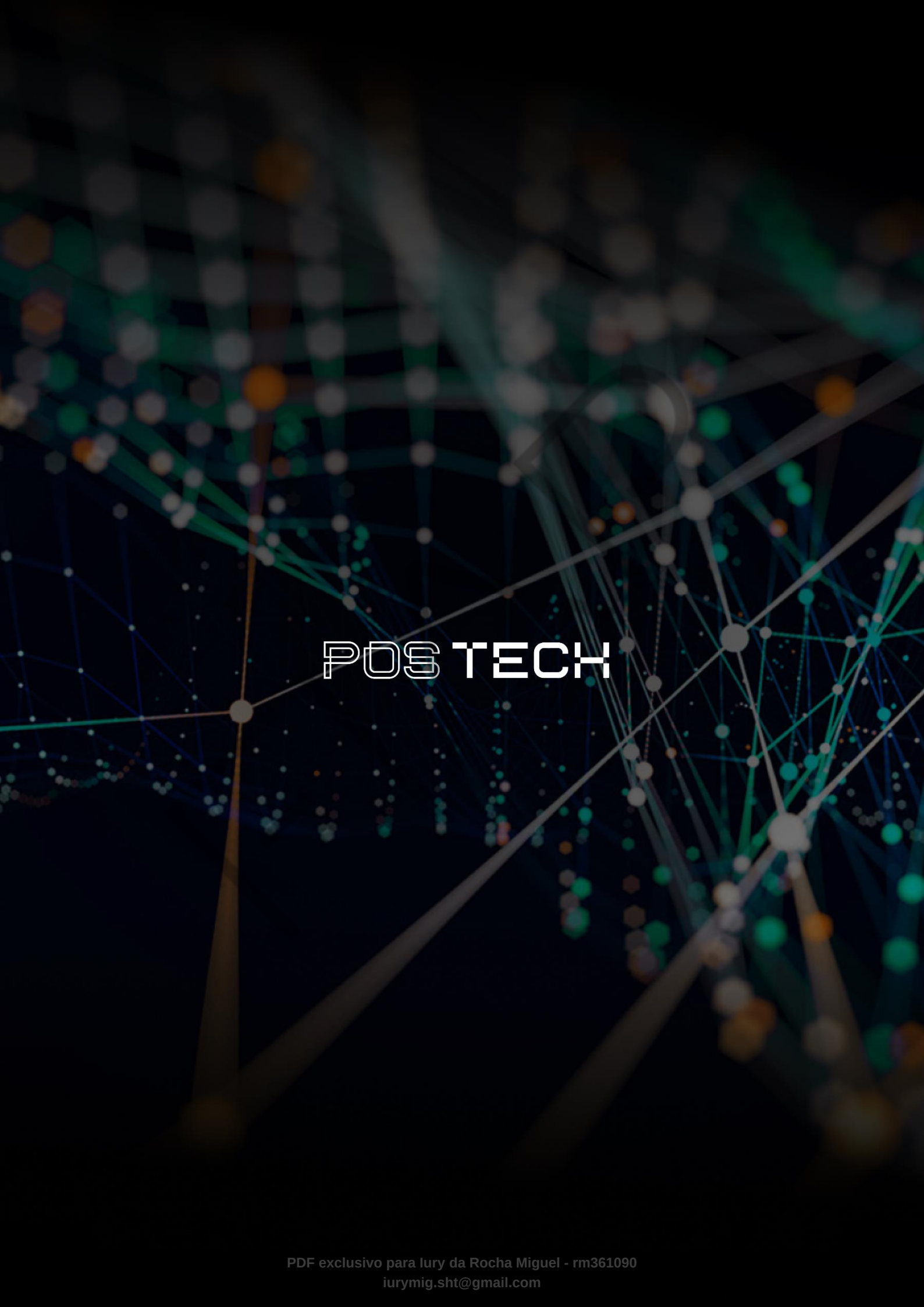
MORGAN, L.; ARUNDEL, J. **Cloud Native DevOps with Kubernetes**. Sebastopol: O'Reilly Media, 2022.

EMANIP

PALAVRAS-CHAVE

DevOps. Containers. Docker. Kubernetes.

EMSE



POSTECH